

ITA0612 – MACHINE LEARNING

CO2-AT1

APPLICATION BASED QUESTIONS

NAME: MONIKA R

REG.NO: 192324281

Submitted To:

Shri Vindhya A

Submitted on:

21-07-2026

QUESTION:1

A Genetic Algorithm is applied to maximize the function $f(x) = x^2$ using an initial population consisting of the values 2, 4, 6, and 8. Determine the fitness value for each individual in the population and select the best candidates based on their fitness scores. Perform a crossover operation by exchanging the least significant bits of the selected individuals to generate new offspring. Additionally, explain the role of mutation in improving the diversity and effectiveness of the solution in Genetic Algorithms.

ANSWER:

Given:

Objective function: $f(x) = x^2$

Initial population: {2, 4, 6, 8}

Step 1: Calculate the Fitness Value

Individual (x)	Fitness ($f(x)=x^2$)
2	4
4	16
6	36
8	64

Step 2: Select the Best Candidates

The individuals with the highest fitness are:

- 8 → Fitness = 64
- 6 → Fitness = 36

These are selected as the parents.

Step 3: Binary Representation

Using 4-bit binary:

- 8 = 1000
- 6 = 0110

Step 4: Crossover (Exchange Least Significant Bit)

Swap the last (least significant) bit of both parents.

Before crossover:

- Parent 1 = **1000**
- Parent 2 = **0110**

The LSB of both is **0**, so after exchanging them:

- Offspring 1 = **1000** (8)
- Offspring 2 = **0110** (6)

Fitness of Offspring

Offspring	Decimal Value	Fitness (x^2)
1000	8	64
0110	6	36

Since the least significant bits are identical, the crossover does **not** produce new individuals in this case.

Role of Mutation in Genetic Algorithms

Mutation is the process of randomly changing one or more bits in a chromosome after crossover.

Example:

- Chromosome **1000** (8)
- Mutate one bit → **1001** (9)

New fitness:

$$f(9) = 9^2 = 81$$

This is better than the previous best fitness (64).

Importance of Mutation

- Introduces new genetic information into the population.
- Maintains diversity among solutions.
- Prevents the algorithm from getting stuck in local optima.
- Helps explore unexplored regions of the search space.

- Increases the chance of finding the global optimum.

INPUT AND OUTPUT:

The image shows a screenshot of a Python IDE with two windows. The left window, titled 'ass2.1.py', contains a Python script for a genetic algorithm. The right window, titled 'IDLE Shell 3.12.4', shows the output of the script.

```

ass2.1.py - C:/Users/monik/AppData/Local/Programs/Python/Python312-32/ass2.1.py (3.12.4)
File Edit Format Run Options Window Help
n = int(input("Enter the population size: "))
population = []
for i in range(n):
    x = int(input(f"Enter individual {i+1}: "))
    population.append(x)
fitness = []
for x in population:
    fitness.append(x * x)
print("\nPopulation and Fitness:")
for i in range(n):
    print("Individual:", population[i], "Fitness:", fitness[i])
combined = list(zip(population, fitness))
combined.sort(key=lambda x: x[1], reverse=True)
parent1 = combined[0][0]
parent2 = combined[1][0]
print("\nSelected Parents:")
print("Parent 1 =", parent1)
print("Parent 2 =", parent2)
b1 = format(parent1, '04b')
b2 = format(parent2, '04b')
print("\nBinary Representation:")
print("Parent 1 =", b1)
print("Parent 2 =", b2)
child1 = b1[:-1] + b2[-1]
child2 = b2[:-1] + b1[-1]
c1 = int(child1, 2)
c2 = int(child2, 2)
print("\nOffspring:")
print("Child 1 =", child1, "Decimal =", c1, "Fitness =", c1*c1)
print("Child 2 =", child2, "Decimal =", c2, "Fitness =", c2*c2)
mutated = list(child1)

```

```

IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
more information.
>>>
= RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python312-32/ass2.1.py =
Enter the population size: 4
Enter individual 1: 2
Enter individual 2: 4
Enter individual 3: 6
Enter individual 4: 8

Population and Fitness:
Individual: 2 Fitness: 4
Individual: 4 Fitness: 16
Individual: 6 Fitness: 36
Individual: 8 Fitness: 64

Selected Parents:
Parent 1 = 8
Parent 2 = 6

Binary Representation:
Parent 1 = 1000
Parent 2 = 0110

Offspring:
Child 1 = 1000 Decimal = 8 Fitness = 64
Child 2 = 0110 Decimal = 6 Fitness = 36

After Mutation:
Mutated Child = 1001 Decimal = 9 Fitness = 81
>>>

```

Ln: 19 Col: 27

Ln: 31 Col: 0

QUESTION:2

A perceptron model is used to classify whether a customer will purchase a product based on two features: advertisement exposure and income level. The weights are 0.5 and 0.3, and the bias is -0.4 . For a customer with advertisement exposure = 2 and income level = 1, compute the net input to the perceptron and apply the step activation function. Determine whether the customer makes a purchase or not.

ANSWER:

Given:

- Advertisement Exposure (x_1) = 2
- Income Level (x_2) = 1
- Weight for Advertisement Exposure (w_1) = 0.5
- Weight for Income Level (w_2) = 0.3
- Bias (b) = -0.4

Step 1: Calculate the Net Input

- The perceptron calculates the net input using the formula:

$$\text{Net Input} = (w_1 \times x_1) + (w_2 \times x_2) + b$$

- Substitute the given values:

$$\begin{aligned} &= (0.5 \times 2) + (0.3 \times 1) + (-0.4) \\ &= 1.0 + 0.3 - 0.4 \\ &= 0.9 \end{aligned}$$

$$\text{Net Input} = 0.9$$

Step 2: Apply the Step Activation Function

The step activation function is:

$$\text{Output} = \begin{cases} 1, & \text{if Net Input} \geq 0 \\ 0, & \text{if Net Input} < 0 \end{cases}$$

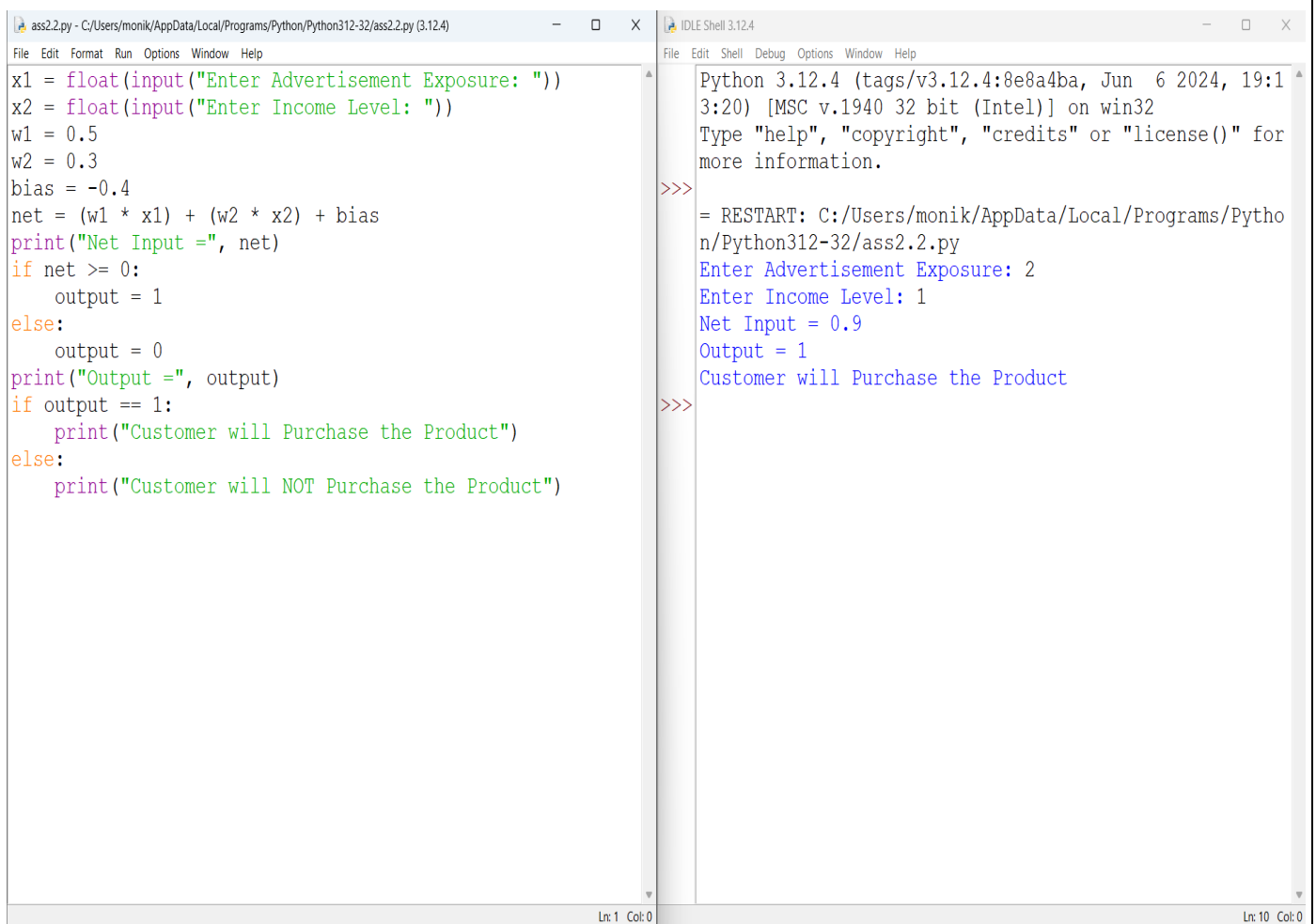
Since, $0.9 \geq 0$

the output is: **1**

Step 3: Decision

- Output = 1 indicates that the customer will purchase the product.
- If the output had been 0, it would indicate that the customer will not purchase the product.

INPUT AND OUTPUT:



The image shows a screenshot of a Python IDE with two windows. The left window, titled 'ass2.2.py', contains the following Python code:

```
x1 = float(input("Enter Advertisement Exposure: "))
x2 = float(input("Enter Income Level: "))
w1 = 0.5
w2 = 0.3
bias = -0.4
net = (w1 * x1) + (w2 * x2) + bias
print("Net Input =", net)
if net >= 0:
    output = 1
else:
    output = 0
print("Output =", output)
if output == 1:
    print("Customer will Purchase the Product")
else:
    print("Customer will NOT Purchase the Product")
```

The right window, titled 'IDLE Shell 3.12.4', shows the execution output:

```
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:13:20) [MSC v.1940 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/monik/AppData/Local/Programs/Python/Python312-32/ass2.2.py
Enter Advertisement Exposure: 2
Enter Income Level: 1
Net Input = 0.9
Output = 1
Customer will Purchase the Product
>>>
```

The status bars at the bottom of the windows indicate 'Ln: 1 Col: 0' for the code editor and 'Ln: 10 Col: 0' for the shell.